Messaging: the live topic for latency and sockets that cannot hurt the node

What was built, how it works and what the runs showed. Branch `feat/messaging`, session of 11 October 2026, steps 4 and 5 of the tuning order in `doc/messaging-architecture.md` section 6.

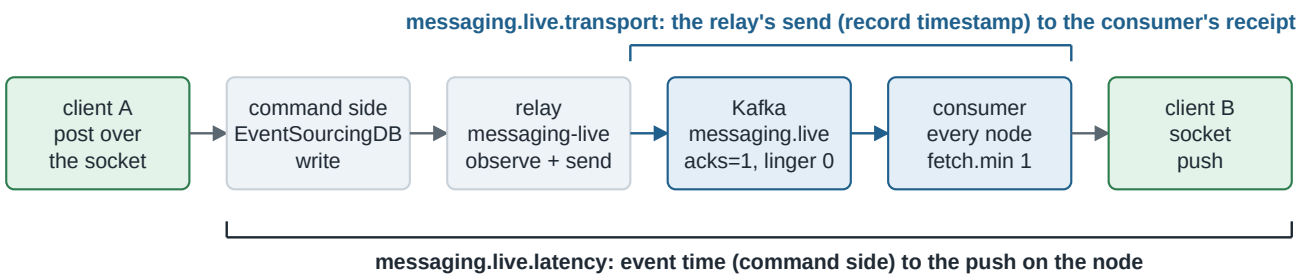
1. Summary

	Before	After
Live topic (step 4)	binder defaults: durable producer (acks=all, idempotent), consumer group named by a random UUID per start, so every restart left a dead group in Kafka	producer and consumer set for latency on the live binding only; the Kafka leg measured on its own: 2 ms at the median, under 10 ms at the 99th percentile, of a 43 ms fan-out; the store's pick-up is 37 ms of it
A client that stops reading (step 5)	the node queued every frame for it without bound and kept the socket for as long as TCP would	at most 256 frames per socket, then close 4408; a dead connection is ended by a 30 s write timeout; ten stalled sockets under a storm of 10,000 bulletins kept 0 frames and were gone within 75 s, against 75,782 frames (about 80 MB) kept forever before
A client that is silent	never noticed	pinged every 30 s, closed 4409 after 75 s without any frame
Posts lost	none	none in any run (1,600 of 1,600 at the standard rate; the 40 posts/s burst saturates the command side, see 5.1)

Both steps are confined to the messaging service: configuration of the live binding, a new `SocketLifecycle` that runs every WebSocket session, a bounded `LiveSink`, a Netty write timeout on the server, and the settings under `messaging.socket`. Nothing is committed; the branch is ready for a commit when you ask.

2. Step 4: the live topic for latency

A message a client posts reaches the other participant's socket through the path of figure 1: the command side writes the event to EventSourcingDB, the relay processor `messaging-live` picks it up from the store's observe stream and sends a record to the Kafka topic `messaging.live`, every node's consumer receives it and pushes a frame to the sockets of that conversation it holds. The whole path was already measured as `messaging.live.latency`; what was missing was the share of Kafka in it. The binder exposes the record's own timestamp (set by the producer at send) as a header of every received message, so the consumer now also records `messaging.live.transport`: the Kafka leg alone.



The relay's projection delay (event written to the relay's handler) is the store's pick-up; latency minus transport minus pick-up is the write and the push.

Figure 1. The fan-out path of one message and the two timers that measure it.

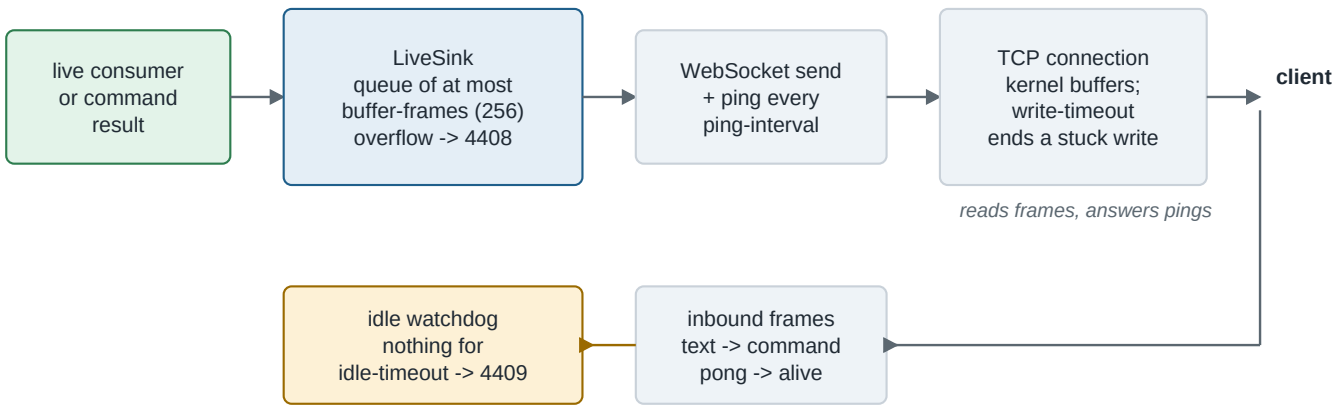
2.1 What was set

Setting	Value	Why
producer `linger.ms`	0	send each record at once, never wait to fill a batch (the binder's default, made explicit)
producer `acks`	1	the partition leader's write is enough; waiting for replicas buys durability the live topic does not need: a lost live record costs one push the client recovers over REST
producer `enable.idempotence`	false	the idempotent producer needs acks=all and keeps sequence numbers per partition; duplicates on the live topic are harmless (clients de-duplicate on the message id)
producer `compression.type`	none	a 300-byte record is not worth compressing
consumer `fetch.min.bytes`	1	the broker answers a fetch as soon as one record is there
consumer `fetch.max.wait.ms`	100	and waits at most this long for one when the topic is quiet (default 500)
consumer `max.poll.records`	100	a poll that returns a burst hands records over in small batches
consumer group	`messaging-live-${ANNOUNCE_NODE}`, `reset-offsets=true`	a stable name per node instead of a random UUID, so a restart leaves no dead group behind; the reset keeps the start at the latest offset, since a node that restarts holds no sockets and has no use for what was published while it was down

The public topic `messaging.v1` keeps the durable defaults: the order context acts on those records.

3. Step 5: sockets that cannot hurt the node

Every WebSocket session has an outbound queue, the `LiveSink`, that the live consumer and the session's own command results push JSON frames into, and that the WebSocket send drains as fast as the client's connection takes bytes. Until now that queue was unbounded and nothing ever timed a socket out. A client that stops reading (a phone in a tunnel, a laptop lid closed, a hostile client) would make the node queue every frame of its conversations for it, forever: heap growth bounded only by the traffic, and a session that is never reclaimed. Figure 2 shows what runs every session now.



4408 and 4409 tell the client why; it reconnects and fetches the messages after the last seq it has over REST.

Figure 2. One socket under SocketLifecycle: the bounded sink, the server ping, the idle watchdog and the write timeout.

3.1 The three policies

- **Bounded sink, close 4408.** The queue holds at most `messaging.socket.buffer-frames` (256) frames. The first frame that does not fit marks the sink as overflowed and completes it: the frames it holds are still delivered, nothing else is queued, and once the send has drained the node closes the socket with 4408 and a reason that says what to do (reconnect, fetch the messages after the last `seq` you have). Closing beats dropping the oldest frames: the client learns that there is a gap instead of silently missing messages. Every refused frame counts in `messaging.sink.dropped{reason=FAIL_OVERFLOW}`.
- **Server ping, idle close 4409.** The node sends a WebSocket protocol ping every `ping-interval` (30 s); browsers and client libraries answer it without application code. A watchdog closes with 4409 a socket from which nothing arrived (the pong counts) for `idle-timeout` (75 s). The watchdog is deliberately not a `timeout` operator on the inbound stream: cancelling the inbound makes Reactor Netty send an empty close frame of its own, and the client would see 1005 instead of the reason. The first version did exactly that and the test caught it.
- **Write timeout.** A peer whose connection stopped taking bytes cannot receive a close frame either: the server's writes to it never complete. Netty's `WriteTimeoutHandler` on every connection of the server (`write-timeout`, 30 s) ends a connection whose write did not complete in time, which detaches the session and frees its sink. Without it, TCP would keep the connection for about 15 minutes of retransmissions.

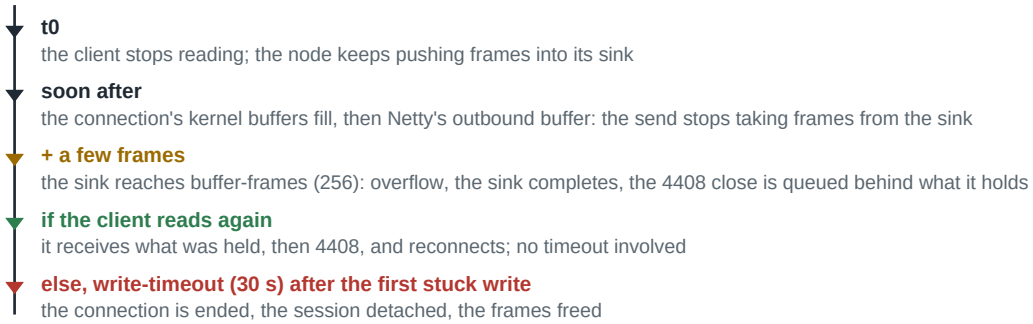


Figure 3. A client that stops reading, over time. A client whose connection is dead (no ACKs) is ended by the write timeout at the first stuck write instead of after TCP's own 15 minutes.

3.2 Two things learned on the way

- **Small frames rarely reach the write timeout.** With 4 KB frames the loopback connection took about 800 frames before the node's send stalled, the sink then held 64 (the test profile's bound), and when the sink overflowed the close frame still fitted into the connection's buffers: the server was done with that socket at once, no timeout needed. The write timeout matters when the connection itself is dead or full, which the test reproduces with 60 KB frames.
- **Count at the source.** A first version of the test derived the number of accepted frames from the drop counter and got three different answers in three runs while the client always received the same 864 frames. The sink now counts what it accepted and what the send took, and the test asserts that every accepted frame was delivered before the close, which holds.

4. Tests

Test	What it proves
`MessagingEndToEndTest`, slow and stalled sockets	A client that reads nothing while 4,000 frames of 4 KB are pushed: the queue never exceeds its bound (64 in the test profile), the frames beyond it are counted as refused, and when the client reads again it receives exactly every frame the sink accepted, then close 4408 with the reason. A client that never reads, pushed 60 KB frames: the session is detached by the write timeout (10 s in the test profile) and nothing stays queued for it.
`MessagingEndToEndTest`, idle socket	A client that sees the pings and leaves them unanswered is closed with 4409 after the idle timeout (4 s in the test profile); a normal client beside it stays open for longer than that and still posts with an ack.
`MessagingEndToEndTest`, meters	`messaging.live.transport` has one sample per live record; the only refusals counted are the slow consumer's.
`LiveSinkTest`	The bounded sink alone: 100 taken, 64 queued, the rest refused; what it accepted is delivered when the consumer resumes, also when merged with a ping stream.
`SocketLifecycleOverNettyTest`	The real `SocketLifecycle` over Spring's Reactor Netty session on a bare server with the write-timeout handler and automatic context propagation, against a client that reads nothing for a while: accepted equals received, close 4408.

The messaging suite passes with 41 tests (13 end to end), 0 failures.

5. Measurements

One node on port 8098 (`ANNOUNCE_NODE=node-a`), a throwaway EventSourcingDB started empty before every run (the development store on port 3000 still hangs on streaming reads), the development Kafka on 9094 and the live MySQL, all on the same 8-core host as the driver. Load as in the earlier sections: 20 support conversations, 40 sockets, 60 s. The host was otherwise idle.

5.1 Step 4: where the fan-out latency is

The A side runs with the binder defaults forced back on the live binding (acks=all, idempotent producer, fetch.max.wait 500 ms, max.poll.records 500); the B side with the settings of section 2.1. Standard load is one post per socket every 3 s (13 posts/s).

Standard load, 800 posts	A: defaults	B: tuned
posts acknowledged / errors	800 / 0	800 / 0
ack latency p50 / p95 / max (driver)	52 / 117 / 177 ms	63 / 212 / 475 ms
`messaging.live.latency` p50 / p95 / p99 (event time to push)	43 / 95 / 142 ms	47 / 167 / 184 ms
relay pick-up: `messaging.projection.delay {messaging-live}` p50 / p95 / p99	37 / 51 / 62 ms	39 / 56 / 62 ms
`messaging.live.transport` p50 / p95 / p99 (the Kafka leg)	2.0 / 3.0 / 7.0 ms	2.0 / 4.0 / 6.0 ms

The Kafka leg is two milliseconds at the median and under ten at the 99th percentile on both sides; the store's observe stream delivers the event to the relay about 37 ms after it was written, and that is where the fan-out latency lives. The A/B difference on the driver side (p95 117 versus 212 ms) is run-to-run noise on a shared host, not the settings: the transport numbers of the two sides are the same. With one broker and replication 1, acks=1 and acks=all are the same wait; the settings are for the production shape.

Burst, one post per socket per second (40 posts/s), 2,400 posts	A: defaults	B: tuned
posts acknowledged / errors	1,477 / 134	1,446 / 111
ack latency p50 / p95 / max (driver)	12.0 / 24.8 / 32.0 s	13.1 / 26.1 / 34.4 s
command latency p50, inside the facade (`messaging.command.latency`)	0.97 s	1.01 s
`messaging.live.latency` p50 / p95 / p99	0.87 / 0.97 / 1.04 s	0.84 / 0.97 / 1.04 s
relay pick-up p50 / p95 / p99	45 / 60 / 70 ms	45 / 64 / 79 ms
`messaging.live.transport` p50 / p95 / p99	2.9 / 5.2 / 9.4 ms	3.0 / 5.2 / 8.1 ms

The burst is a finding of its own, not a Kafka one. At 40 posts a second one node's command path saturates: a post takes about a second inside the facade, the driver's open loop keeps sending, the acks queue up to tens of seconds and about 5 % of the posts end in an error (retries exhausted or the driver's timeout). The relay and Kafka are untouched by it: the pick-up stays at 45 ms and the transport at 3 ms; the 0.87 s of live latency is the time the event spent waiting to be written, since the event time is taken before the write. The same 40 sockets at 13 posts/s had every post acknowledged in 52 ms. The command side's throughput ceiling (somewhere between 13 and 40 posts/s on this host, with 40 commands in flight) is the next thing to measure and is added to the tuning order as step 8; a first run at ten times the standard rate, 133 posts/s, had acknowledged 1,329 of 7,994.

5.2 Step 5: a node with sockets that do not read

`StalledSocketDriver` opens eleven staff dashboard sockets on the node: one that reads and ten whose client never reads (and, with `handlePing` on, never answers a ping), then posts 5,000 bulletins of 2,000 characters over REST at a concurrency of 16. Every bulletin goes through the live topic to every staff socket: 55,000 frames of about 2.1 KB, 50,000 of them for sockets that will never take them. Two configurations of the same node.

	Unbounded (buffer-frames 0, no timeouts: the state before step 5)	Bounded (256 frames, ping 30 s, idle 75 s, write timeout 30 s)
bulletins posted	9,998 in 163 s	9,998 in 163 s
frames queued at the end of the burst (`sink.buffered`, deepest socket)	75,782 in all, 7,615 on the deepest socket; still there a minute later	0 (never above 256 per socket)
frames refused (`sink.dropped`)	0	22,633 (reason FAIL_OVERFLOW)
heap used, before -> after the burst	about 80 MB of frame strings retained for the ten sockets (75,782 x 1.1 KB), growing with every bulletin, never released	nothing retained for the stalled sockets
staff sockets on the node after the burst	11: the ten stalled ones stay until TCP gives up	1 (the reader); the ten stalled sockets were detached about 75 s after they connected, by the idle timeout, since they answer no ping; the write timeout would have followed within seconds
the reading socket	received every bulletin	received every bulletin

The two nodes received the same storm. The unbounded one, which is the state before this step, ends it holding 75,782 frames for ten clients that will never read them and keeps the ten sessions until the operating system gives up on the connections; every further bulletin adds ten more frames. The bounded

one refuses what the ten clients cannot take, counts it, and is rid of them within the idle timeout, with nothing left in memory; the socket that reads receives every bulletin either way. The heap figure of the Prometheus scrape is not shown: without a forced collection it only measures garbage, and both nodes grew it to about 600 MB during the burst.

6. Operating notes

- **Choosing the bound.** 256 frames is about 80 KB of JSON for messages, a minute of a very busy conversation. A client that falls that far behind is better served by a reload over REST than by a backlog on the socket. Raise it for staff dashboards that receive bulletin storms if 4408 closes show up in their logs; `messaging.sink.buffered.max` shows how deep a healthy socket ever gets.
- **The three timers.** `ping-interval` below `idle-timeout` by a factor of at least two, so one lost pong does not close a healthy socket; `write-timeout` longer than the slowest honest client needs to drain one frame, 30 s is generous. Zero turns each of them off.
- **Clients.** On 4408 and 4409 reconnect and call `GET /api/conversations/{id}/messages?after=<last seq>`; nothing is lost from the conversation, only from the socket. Clients that use the application-level `{"type":"ping"}` frame keep working; the protocol ping needs nothing from them.
- **The write timeout applies to every connection of the server**, REST included: an HTTP client that stops reading a response for 30 s loses it. No endpoint of this service streams anything long enough for that to matter.
- **The Kafka settings are per binding.** `messaging.v1` keeps acks=all and the idempotent producer. With a single broker and replication 1 on the development stack the producer settings cannot show a difference; they are set for the production shape, where acks=all waits for replicas on another host.
- **Consumer groups.** `reset-offsets=true` also re-seeks on every partition assignment. A single-member group only rebalances at start, so nothing is skipped in practice; if the live topic ever gets several consumers per node, drop the reset and keep the random name instead.
- **Where the latency is.** The store's observe stream delivers an event to the relay about 40 ms after it was written (the relay's projection delay), Kafka adds two. Pushing to local sockets straight from the command side would remove both for sockets on the posting node and is the next thing to try if 40 ms ever matters; it would need duplicate suppression by message id on that node.

7. Files changed

Paths under `messaging/src/main/java/org/linlinjava/announce/messaging/` unless noted.

File	Change
interfaces/ws/SocketLifecycle.java	new: runs every session: outbound sink + server ping, inbound text frames one at a time, idle watchdog (4409), slow-consumer close (4408)
infrastructure/live/LiveSink.java	bounded queue, overflowed flag, accepted/emitted counters
infrastructure/live/LiveSessions.java	bound from messaging.socket.buffer-frames; sinksOf(conversation) for inspection
infrastructure/live/LiveEventsSubscriber.java	messaging.live.transport from the record timestamp header
infrastructure/metrics/MessagingMeters.java	LIVE_TRANSPORT
infrastructure/configuration/SocketConfig.java	new: Netty WriteTimeoutHandler on every server connection
application/MessagingProperties.java	Socket settings (buffer-frames, ping-interval, idle-timeout, write-timeout)
interfaces/ws/ConversationWebSocketHandler.java, StaffWebSocketHandler.java	delegate the session to SocketLifecycle
src/main/resources/config/application.yml	live binding producer/consumer settings, stable consumer group with reset
src/test/resources/application-test.yml	socket settings small enough for the suite
test: MessagingEndToEndTest.java	StalledClient; slow/stalled and idle socket cases; transport and drop assertions
test: infrastructure/live/LiveSinkTest.java, SocketLifecycleOverNettyTest.java	new unit and bare-Netty tests
test: load/StalledSocketDriver.java	new: stalled staff sockets under a bulletin burst against a running node
doc/messaging-architecture.md, doc/messaging-context.md	sections 2.6, 4.3, 5, 6 and the new 7d; context sections 5 and 7

Branch state: all of the above is uncommitted on `feat/messaging`, on top of this morning's leader-election commit. Next candidate in the tuning order: step 7 (inbox query batching); the direct local push of the operating notes is the follow-up to step 4.